
第 6 章

Networkx 及其基本使用

本章配套：正文小节 / 上机 lab / 练习 quiz / 常见误区与技巧

6. NetworkX 及其基本使用（新版讲义）

本页是第 6 章的章首页（索引）。正文位于 01-创建图.md ~ 05-常见误区与技巧.md，与原版资料完全分离（原版见../原始资料/chap6/）。

本章概览

复杂系统（社交网络、交通网、生物分子网络、互联网）都可抽象为“节点 + 边”的图（Graph）。NetworkX 是 Python 中创建、操作、分析复杂网络的标准库：它内置 Graph（无向图）与 DiGraph（有向图），以及度、聚类系数、中心性、最短路径、最小生成树、社区划分等大量图论算法。本章将带你从“创建一张图”走到“分析图的结构指标”，再到“求解遍历、最短路径、最小生成树等基本问题”。

学习目标

学完本章，你应该能够：

1. 用 nx.Graph 与 nx.DiGraph 创建无向图/有向图，并添加节点、边与属性（权重、标签）。
2. 读取 nodes / edges / degree，访问邻居、前驱、后继与入度/出度。
3. 计算并解释度/度分布、平均路径长度、聚类系数、传递性、介数中心性、接近中心性、模块度等结构指标。
4. 用 DFS / BFS 遍历图，并用 shortest_path 求无权/带权最短路径（Dijkstra、Bellman-Ford、Floyd-Warshall）。
5. 用 minimum_spanning_tree 求解最小生成树，并理解 Kruskal / Prim 的适用场景。
6. 判断图的连通/强连通/弱连通，识别连通分量。
7. 完成一个综合案例（空手道俱乐部的社区划分与最短路径），把本章知识串起来。

案例索引

案例卡	所在节	技术点	难度
1. 用 nx.Graph 建“班级社交网”——节点/边/属性/度	01 创建图	Graph、add_nodes_from 属性、add_weighted_edges_from、度/点强度	★☆☆
2. 中心性分析：谁是班里最重要的人（度/介数/接近）	02 分析图	degree、betweenness、closeness、排序比较	★★☆
3. 加权最短路径：地铁换乘总时间最少路线	03 求解图的基本问题	加权图、dijkstra_path、single_source、floyd_warshall	★★☆
4. 空手道俱乐部——创建/指标/社区/最短路/可视化	04 综合案例	度分布、聚类/传递性、greedy_modularity、modularity、shortest_path	★★★

先修要求与运行环境

- 熟悉 Python 基础语法（变量、列表、字典、函数、循环）。
- 建议先学第 1 章 NumPy（用于邻接矩阵、向量化统计）。
- 安装 Python 3.10+ 与 NetworkX：

```
pip install networkx
# 若需要绘图/上机, 再安装
pip install matplotlib numpy
```

- 推荐在 JupyterLab / VS Code 中打开.ipynb 练习；命令行列出的脚本直接保存为.py 运行即可。

本章目录

小节	文件	内容
01 创建图	01-创建图.md	Graph/DiGraph、节点边添加、属性、生成器、邻接矩阵、案例卡 1（班级社交网）
02 分析图	02-分析图.md	度/度分布、平均路径、聚类系数、传递性、中心性、连通性、模块度、案例卡 2（中心性）
03 求解基本问题	03-求解图的基本问题.md	DFS/BFS、最小生成树、Dijkstra/Bellman-Ford/Floyd-Warshall、案例卡 3（加权最短路）
04 综合案例	04-综合案例.md	空手道俱乐部：度分布、社区划分、最短路径（图文案例，含案例卡 4）
05 常见误区与技巧	05-常见误区与技巧.md	易错点清单、性能技巧、调试建议、自测清单

数学预备与附录

本章的图论背景已集中到附录《数学与算法补充》：

- **D 图论与网络**：D.1 图表示、D.2 基本量、D.3 经典算法（BFS/最短路/MST/中心性/社区）、D.4 NetworkX 对应；
 - [打开附录 D](#) | [附录索引](#)
- 建议先读 D.1~D.3 再进正文；做最短路/社区任务时回查 D.3。

练习与上机入口

- [本章练习 \(exercises/\)](#)：自测 quiz (10 题)、作业题、参考答案。
- [本章上机 \(lab/\)](#)：循序渐进的上机 notebook，含环境自检、逐点演练与综合任务。
- [本章参考与延伸阅读 \(references.md\)](#)：官方文档、精品教程、习题实战、中文资料。
- [教学说明 \(teaching.md\)](#)：课时安排、重点难点、考核建议（教师用）。

建议课时

环节	学时	对应内容
讲课	3-4 学时	01-03 正文 + 04 案例讲解
上机	2-3 学时	lab/ 逐题完成；课后完成 exercises/ 作业

使用说明

- **学生**：先读 01–03 正文并运行代码 → 完成 lab 上机 → 做 exercises 自测与作业 → 用 04 案例串联知识。遇到不熟悉的函数时，到 references.md 查官方对应章节。
- **教师**：按 teaching.md 的课时表讲；lab 可作为上机课内容；exercises 中的 quiz 可回收批改。

01 创建图：无向图、有向图、属性与生成器

本节对应原版 6.1 的内容，并增补学习目标、常见误区、思考题与延伸阅读。配套图：images/graph_basic.png（由 build/make_chap6_figures.py 生成）。

本节目标

- 理解 NetworkX 中图对象（Graph / DiGraph）的基本概念；
- 会创建空图并添加节点、边；
- 会读取 nodes / edges / degree，访问邻居、前驱、后继与入度/出度；
- 会给节点和边添加属性（权重、标签）；
- 会用常用图生成器（path_graph、cycle_graph、complete_graph、grid_2d_graph、karate_club_graph 等）快速造图；
- 会用 nx.to_numpy_array 把图转成邻接矩阵，供 NumPy 使用。

先修

- Python 列表、元组、字典、for 循环；了解函数与类的基本概念。
-

1.1 NetworkX 简介与安装

NetworkX 是 Python 生态中创建、操作、分析复杂网络的库。它是一个纯 Python 实现（底层可结合 NumPy/SciPy），接口直观，覆盖图论与网络科学的常用算法。

安装（建议 Python 3.10+）：

```
pip install networkx
```

导入并查看版本：

```
import networkx as nx
print(nx.__version__)
```

3.3

说明：本章代码在 NetworkX 3.x 下验证；不同小版本的个别函数参数可能变化，遇到不熟悉的接口请以官方文档为准。

1.2 创建无向图（Graph）

无向图的边没有方向，用 nx.Graph 创建。

```
import networkx as nx
```

```
# 创建一个空的无向图
```

```
G = nx.Graph()
```

```
# 添加节点
```

```
G.add_node(1)
```

```
G.add_nodes_from([2, 3, 4])
```

```
# 添加边
```

```
G.add_edge(1, 2)
```

```
G.add_edges_from([(2, 3), (3, 4), (4, 1)])
```

```
# 查看节点与边
```

```
print(G.nodes())
```

```
print(G.edges())
```

```
[1, 2, 3, 4]
```

```
[(1, 2), (1, 4), (2, 3), (3, 4)]
```

说明: `add_node` 加单个节点; `add_nodes_from` 加一个可迭代集合 (列表、元组、`range`); `add_edge` 加单条边; `add_edges_from` 加多条边。 `G.nodes()` 与 `G.edges()` 返回的是视图对象, 可转成 `list` / `tuple` 使用。

1.2.1 度 (Degree) 与邻居

无向图中, 节点的度是与它相连的边数。

```
print(G.degree())          # 所有节点的度
for node, degree in G.degree():
    print(f"Node {node} has degree {degree}")
print(list(G.neighbors(1))) # 节点 1 的所有邻居
```

```
[(1, 2), (2, 2), (3, 2), (4, 2)]
```

```
Node 1 has degree 2
```

```
Node 2 has degree 2
```

```
Node 3 has degree 2
```

```
Node 4 has degree 2
```

```
[2, 4]
```

1.2.2 连通性与基本遍历

```
print(nx.is_connected(G))          # 是否连通
print(list(nx.dfs_preorder_nodes(G, source=1))) # 深度优先遍历顺序
print(list(nx.bfs_edges(G, source=1)))          # 广度优先遍历的边
```

```
True
```

```
[1, 2, 3, 4]
```

```
[(1, 2), (1, 4), (2, 3)]
```

1.2.3 绘图

```
import matplotlib.pyplot as plt
```

```
# 使用 matplotlib 绘制图
```

```
nx.draw(G, with_labels=True, font_weight="bold")
```

```
plt.show()
```

运行后得到一张带标签的环形布局图。本章的 `images/graph_basic.png` 由脚本生成, 左图为无向图、右图为有向图。

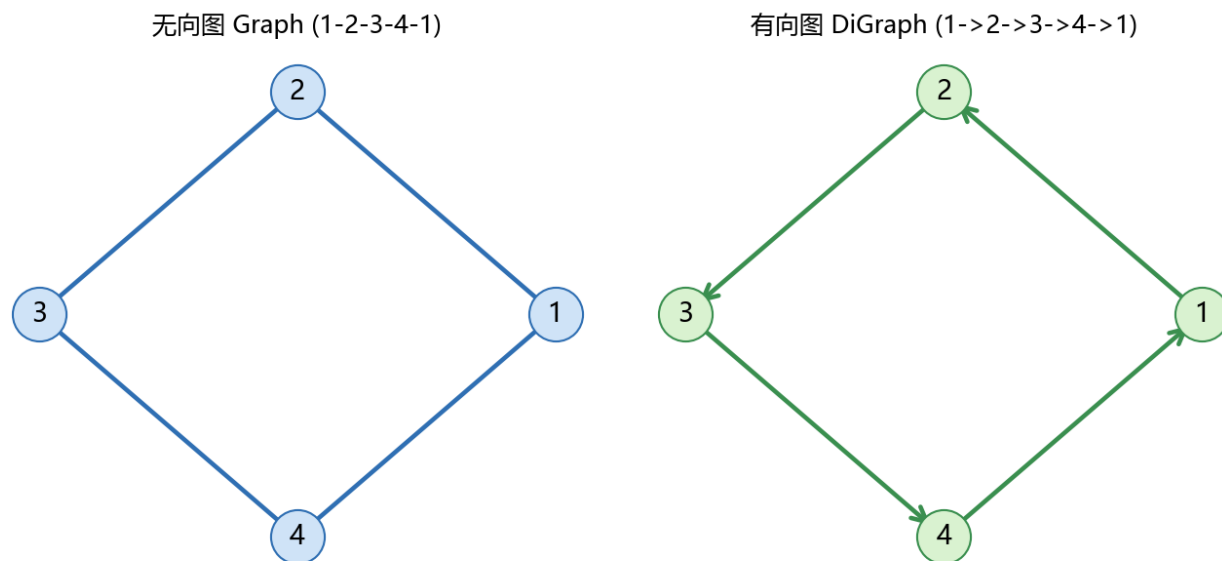


Figure 1: 无向图与有向图示例

1.3 创建有向图 (DiGraph)

有向图的边有方向，用 `nx.DiGraph` 创建。

```
import networkx as nx
```

```
DG = nx.DiGraph()
DG.add_node(1)
DG.add_nodes_from([2, 3, 4])
DG.add_edge(1, 2)
DG.add_edges_from([(2, 3), (3, 4), (4, 1)])
```

```
print(DG.nodes())
print(DG.edges())
```

```
[1, 2, 3, 4]
[(1, 2), (2, 3), (3, 4), (4, 1)]
```

1.3.1 入度、出度、前驱与后继

```
print(DG.in_degree())
print(DG.out_degree())
for node in DG.nodes():
    print("Node", node, "in-degree", DG.in_degree(node), "out-degree", DG.out_degree(node))
print(list(DG.predecessors(2))) # 指向 2 的节点
print(list(DG.successors(2)))  # 2 指向的节点

[(1, 1), (2, 1), (3, 1), (4, 1)]
[(1, 1), (2, 1), (3, 1), (4, 1)]
Node 1 in-degree 1 out-degree 1
Node 2 in-degree 1 out-degree 1
```

```
Node 3 in-degree 1 out-degree 1
Node 4 in-degree 1 out-degree 1
[1]
[3]
```

注意：原始资料里用 `zip(nodes, in_degree, out_degree)` 的写法会把元组拆错 (`in_degree` 返回的是 (节点, 度数) 对)。建议像上面这样用 `DG.in_degree(node)` 单独取度数, 或用 `dict(DG.in_degree())`。

1.3.2 强连通与弱连通

有向图的连通性要区分：

- 强连通：任意两点都互相可达；
- 弱连通：忽略方向后，任意两点可达。

```
print(nx.is_strongly_connected(DG))
print(nx.is_weakly_connected(DG))

True
True
```

1.3.3 有向图遍历

```
print(list(nx.dfs_preorder_nodes(DG, source=1)))
print(list(nx.bfs_edges(DG, source=1)))

[1, 2, 3, 4]
[(1, 2), (2, 3), (3, 4)]
```

绘图时 NetworkX 会根据有向性画箭头：

```
import matplotlib.pyplot as plt
nx.draw(DG, with_labels=True, font_weight="bold", arrowstyle="->", arrowsize=20)
plt.show()
```

1.4 节点与边属性、加权图

1.4.1 权重 (weight)

```
G = nx.Graph()
G.add_weighted_edges_from([("A", "B", 10), ("A", "C", 6), ("B", "C", 5)])
print(list(G.edges(data=True)))
print(G[("A", "B")])          # 取出 A-B 边的属性字典
print(G.degree())             # 普通度
print(G.degree(weight="weight")) # 加权度 (点强度)

[('A', 'B', {'weight': 10}), ('A', 'C', {'weight': 6}), ('B', 'C', {'weight': 5})]
{'weight': 10}
[('A', 2), ('B', 2), ('C', 2)]
[('A', 16), ('B', 15), ('C', 11)]
```

点强度 (weighted degree) 是节点所有相连边的权重和：A 连 B(10)、C(6)，所以是 16。加权网络上“重要节点”的衡量常从点强度出发。

1.4.2 节点属性

```
G.add_node("A", role="hub", color="red")
print(list(G.nodes(data=True)))
```

```
[('A', {'role': 'hub', 'color': 'red'}), ('B', {}), ('C', {})]
```

节点/边上可以挂任意字典属性；用 `G.nodes(data=True)` 与 `G.edges(data=True)` 一起查看。

1.5 常用图生成器

NetworkX 自带许多“造图”模板，适合课堂演示与随机网络实验。

```
print(list(nx.path_graph(6).edges()))
print(list(nx.cycle_graph(5).edges()))
print(list(nx.complete_graph(4).edges()))
print(list(nx.star_graph(5).edges()))
print(list(nx.grid_2d_graph(2, 3).nodes()))
print(list(nx.grid_2d_graph(2, 3).edges()))
K = nx.karate_club_graph()
print("karate_club:", K.number_of_nodes(), K.number_of_edges())
```

```
[(0, 1), (1, 2), (2, 3), (3, 4), (4, 5)]
[(0, 1), (0, 4), (1, 2), (2, 3), (3, 4)]
[(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
[(0, 1), (0, 2), (0, 3), (0, 4), (0, 5)]
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2)]
[((0, 0), (1, 0)), ((0, 0), (0, 1)), ((0, 1), (1, 1)), ((0, 1), (0, 2)), ((0, 2), (1, 2)),
karate_club: 34 78
```

随机网络生成器通常带 `seed` 参数，保证结果可复现：

```
er = nx.erdos_renyi_graph(10, 0.3, seed=42)
ws = nx.watts_strogatz_graph(10, 4, 0.1, seed=42)
ba = nx.barabasi_albert_graph(10, 2, seed=42)
print("erdos_renyi:", er.number_of_nodes(), er.number_of_edges())
print("watts_strogatz:", ws.number_of_nodes(), ws.number_of_edges())
print("barabasi_albert:", ba.number_of_nodes(), ba.number_of_edges())
```

```
erdos_renyi: 10 17
watts_strogatz: 10 20
barabasi_albert: 10 16
```

1.6 邻接矩阵 (与 NumPy 对接)

用 `nx.to_numpy_array` 可把图转成邻接矩阵，便于交给 NumPy/SciPy 做数值计算。

```
import numpy as np
G = nx.Graph()
G.add_edges_from([(1, 2), (2, 3), (3, 1), (3, 4)])
```

```
A = nx.to_numpy_array(G)
print(A)
[[0. 1. 1. 0.]
 [1. 0. 1. 0.]
 [1. 1. 0. 1.]
 [0. 0. 1. 0.]]
```

有向图会得到非对称矩阵：

```
D = nx.DiGraph(); D.add_edges_from([(1, 2), (2, 3), (3, 1), (2, 4)])
print(nx.to_numpy_array(D))
[[0. 1. 0. 0.]
 [0. 0. 1. 1.]
 [1. 0. 0. 0.]
 [0. 0. 0. 0.]]
```

案例卡 1：用 nx.Graph 建“班级社交网”——节点/边/属性/度

目标

把 5 位同学当成“节点”，把“一起学习/讨论”当成“边”，用 nx.Graph 建一张班级社交网。你会顺带掌握：给节点挂属性、给边加权重（每周讨论次数）、看每个节点的度（直接朋友数）与点强度（加权度 = 朋友往来总次数）。

代码

```
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np
import os

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
np.set_printoptions(precision=4, suppress=True)
```

1) 建空图

```
G = nx.Graph()
```

2) 一次加多个节点，并给每个节点挂属性（班级/兴趣）

```
students = [
    ('小明', {'cls': '一班', 'interest': '数学'}),
    ('小红', {'cls': '一班', 'interest': '编程'}),
    ('小刚', {'cls': '一班', 'interest': '物理'}),
    ('小丽', {'cls': '二班', 'interest': '语文'}),
    ('小华', {'cls': '二班', 'interest': '美术'}),
]
G.add_nodes_from(students)
```

3) 加边, weight 表示每周一起讨论的次数

```

G.add_weighted_edges_from([
    ('小明', '小红', 3),
    ('小明', '小刚', 2),
    ('小红', '小刚', 4),
    ('小刚', '小丽', 1),
    ('小丽', '小华', 5),
])

print('节点数:', G.number_of_nodes(), ' 边数:', G.number_of_edges())
print('节点及属性:')
for n, data in G.nodes(data=True):
    print(' ', n, data)
print('边及权重:')
for u, v, w in G.edges(data='weight'):
    print(f' {u} -- {v} : {w}')

# 4) 度 与 点强度 (加权重)
print('每个节点的度 (降序):', sorted(dict(G.degree()).items(), key=lambda x: (-x[1], x[0])))
print('每个节点的加权重/点强度 (降序):',
      sorted(dict(G.degree(weight='weight')).items(), key=lambda x: (-x[1], x[0])))

# 5) 画出来 (jupyter 里可直接显示; 脚本环境保存到 images/)
os.makedirs('images', exist_ok=True)
pos = nx.spring_layout(G, seed=42, k=0.8)
plt.figure(figsize=(6.4, 4.6))
nx.draw_networkx_nodes(G, pos, node_size=800, node_color='#cfe3f7', edgecolors='#2f6fb3')
nx.draw_networkx_edges(G, pos, width=2.0, edge_color='#4b8bbe')
nx.draw_networkx_labels(G, pos, font_size=11, font_family='Microsoft YaHei')
plt.title('班级社交网')
plt.axis('off')
plt.tight_layout()
plt.savefig('images/case1_class_network.png', dpi=150)
print('已保存 images/case1_class_network.png')
plt.close()

```

运行结果 (已运行核验)

节点数: 5 边数: 5

节点及属性:

```

小明 {'cls': '一班', 'interest': '数学'}
小红 {'cls': '一班', 'interest': '编程'}
小刚 {'cls': '一班', 'interest': '物理'}
小丽 {'cls': '二班', 'interest': '语文'}
小华 {'cls': '二班', 'interest': '美术'}

```

边及权重:

```

小明 -- 小红 : 3
小明 -- 小刚 : 2
小红 -- 小刚 : 4
小刚 -- 小丽 : 1

```

小丽 -- 小华 : 5

每个节点的度 (降序): [('小刚', 3), ('小丽', 2), ('小明', 2), ('小红', 2), ('小华', 1)]

每个节点的加权重/点强度 (降序): [('小刚', 7), ('小红', 7), ('小丽', 6), ('小华', 5), ('小明', 5)]

已保存 images/case1_class_network.png

讲解

一句话解释: 先画一张 “人 = 点、一起学习 = 连线” 的网; 每条线还能记一个数 (每周讨论几次), 这样不但知道 “谁朋友多”, 还知道 “谁和朋友来往最勤”。

1. `G = nx.Graph()` 建一张**无向图**: 朋友关系不分方向, A 和 B 是朋友就等于 B 和 A 是朋友。
2. `G.add_nodes_from(students)` 里每个元素是 (节点名, 属性字典), 所以一次就能把 “班级、兴趣” 挂在节点上; 查看时用 `G.nodes(data=True)` 会得到 (节点, 属性字典) 对。
3. `G.add_weighted_edges_from(...)` 的每个元素是 (起点, 终点, 权重); `weight` 是 NetworkX 约定好的边属性名, 后面 `degree(weight='weight')` 才能自动读到它。
4. `G.degree()` 是**直接邻居数**: 小明连小红、小刚, 所以是 2; 小刚连小明、小红、小丽, 是 3。
5. `G.degree(weight='weight')` 是把 “每条边的权重相加”, 等于**点强度**: 小刚 = $2+4+1=7$, 小红 = $3+4=7$, 小丽 = $1+5=6$, 小华 = 5, 小明 = $3+2=5$ 。所以小刚虽然度最高, 但 “来往总次数” 第一的是小刚和小红并列。
6. `nx.spring_layout(G, seed=42)` 给节点随机选初始位置, 固定 `seed=42` 才能每次画出同一张布局; 保存到 images/ 前先 `os.makedirs('images', exist_ok=True)`, 避免目录不存在报错。

班级社交网

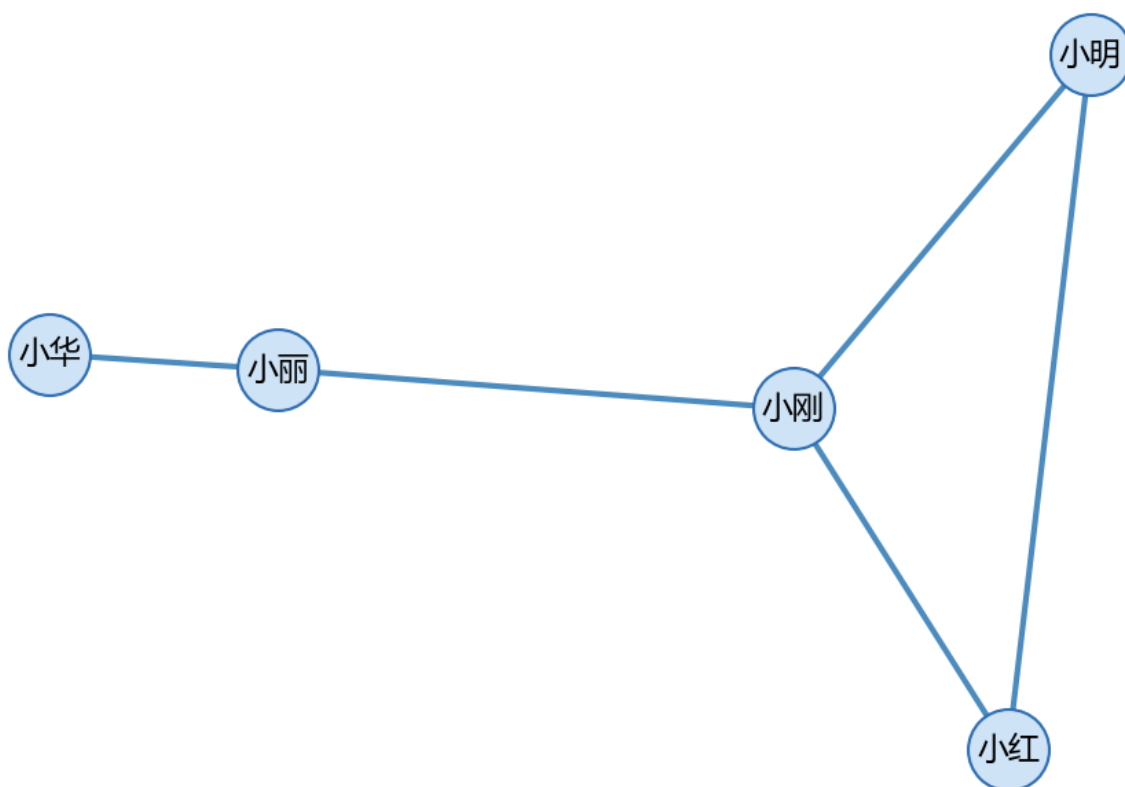


Figure 2: 班级社交网

主要用法 / API

需求	写法	说明
建空无向图	<code>nx.Graph()</code>	边无方向
加单个/多个节点	<code>add_node</code> / <code>add_nodes_from</code>	可带属性
加单条/多条边	<code>add_edge</code> / <code>add_edges_from</code>	无向图自动补反向
加带权边	<code>add_weighted_edges_from([(u, v, w)])</code>	属性名 <code>weight</code>
查看节点/边属性	<code>G.nodes(data=True)</code> / <code>G.edges(data=True)</code>	得到 (节点, dict) / (u, v, dict)
度	<code>G.degree()</code> / <code>G.degree(node)</code>	直接邻居数
点强度	<code>G.degree(weight='weight')</code>	加权度
画图	<code>nx.draw_networkx_*</code> + <code>plt.savefig</code>	需配中文字体

常见错误

- 1. 以为 `add_edge(u, v, 10)` 的第三个参数是权重 → 错误; 正解是 `add_edge(u, v, weight=10)` 或 `add_weighted_edges_from`.
- 2. 忘记 `G.degree(weight='weight')` 的 `weight` 参数名要跟边属性名一致; 如果边属性叫 `time`, 就要写 `weight='time'`.
- 3. 用 `plt.show()` 在无界面环境会卡住/无反应 → 脚本/作业里用 `plt.savefig(...)`; `spring_layout` 不传 `seed` 每次布局都不同, 作业截图容易对不上.
- 4. `sorted(G.degree())` 返回的是按节点名排列的 (节点, 度) 列表; 想按度数排序要 `key=lambda x: (-x[1], x[0])`.

拓展 / 跟练

- 1. 新加一位同学 小强 与 小华 连一条权重 2 的边, 再打印度与点强度, 看谁变成 “朋友最多”.
- 2. 把每条边的权重改成 “共同爱好数”, 分别打印度与点强度, 观察两者排名是否一致.
- 3. 用 `nx.to_numpy_array(G, nodelist=sorted(G.nodes()), weight='weight')` 得到邻接矩阵, 验证它是对称矩阵.
- 4. 给每个节点补一个 `age` 属性, 然后筛选出 “一班” 的学生 (答案: `[n for n, d in G.nodes(data=True) if d.get('cls')=='一班']`).

常见误区

- 1. 把 **nodes/edges** 当普通列表: 它们是视图对象, 直接判断成员/转 list 均可, 但不要对它做破坏性修改 (例如 `G.nodes()` 不能赋值).
- 2. 在有向图上误用 **neighbors**: 有向图有前驱与后继两个方向, 要用 `predecessors` / `successors`, `neighbors` 在有向图中返回后继节点.
- 3. 误用 **zip(nodes, in_degree, out_degree)**: `in_degree` 返回的是 (节点, 度数) 对, 直接解包会把元组整体当成 “度数”. 建议用 `DG.in_degree(node)` 或 `dict(DG.in_degree())`.
- 4. 忘记给加权边指定 **weight** 关键字: `add_weighted_edges_from` 默认用 `weight` 作为属性名; 若用 `add_edge` 加权重需写成 `add_edge(u, v, weight=w)`.
- 5. 随机图不设 **seed**: 结果每次不同, 课堂/作业演示请设 `seed` (如 `seed=42`).

6. **把图当作“可哈希容器”直接比较**：两个结构相同的图不一定相等，判断是否同构要用 `nx.is_isomorphic`。

思考题

1. 无向图 `G.nodes()` 返回的节点顺序是添加顺序还是自动排序？用循环访问时是否稳定？
2. 为什么强连通要求“互相可达”？给出一个弱连通但不强连通的有向图例子。
3. `nx.Graph(G)` 与 `nx.DiGraph(G)` 把一个无向图转成有向图时，边会变成几条？

动手练习（详见 lab）

- 创建一个 6 节点的无向环图 (`cycle_graph`)，打印每条边与每个节点的度；
- 创建一个包含 5 个节点的星形图 (`star_graph`)，找出中心节点并计算其度；
- 给一个加权图添加节点属性，打印 `nodes(data=True)` 与 `edges(data=True)`；
- 用 `karate_club_graph` 生成图并转为邻接矩阵，验证矩阵是 34x34 且对称。

延伸阅读

- NetworkX 官方教程（图基础）：[链接](#)
- NetworkX 官方图类型参考：[链接](#)
- NetworkX 官方生成器列表：[链接](#)

02 分析图：结构指标

本节对应原版 6.2 的内容。先给指标下定义，再用 NetworkX 计算，并给出真实运行结果。

本节目标

- 理解度、度分布、平均路径长度、聚类系数、传递性、介数中心性、接近中心性、模块度等指标的含义；
- 会用 NetworkX 计算这些指标并读懂结果的数值含义；
- 能判断无向图是否连通，有向图是否强连通/弱连通，并提取连通分量；
- 会计算加权图的点强度。

先修

- 第 01 节：创建 Graph / DiGraph，访问 nodes / edges / degree。

2.1 指标一览

指标	名字	直观含义	反映什么
度	Degree	与节点直接相连的边数	节点的“直接影响力”
度分布	Degree Distribution	不同度数的节点占比	网络拓扑类型（均匀/无标度）
平均路径长度	Average Path Length	任意两点最短路径边数的均值	网络的整体“紧密度/传播效率”
聚类系数	Clustering Coefficient	一个节点的邻居彼此相连的程度	局部集聚、三角关系
传递性	Transitivity	全网三角形与“开放三元组”的比值	全局成团倾向
介数中心性	Betweenness Centrality	经过该点的最短路径占比	节点作为“桥梁/瓶颈”的重要性
接近中心性	Closeness Centrality	到其他节点的平均距离越短越接近	节点的“可达性/中心程度”
模块度	Modularity	社区划分质量	网络是否存在明显社区结构
点强度	Node Strength	加权度：相连边权重之和	加权网络中的“重要性”

2.2 节点度与度分布

2.2.1 度

```
import networkx as nx

G = nx.Graph()
G.add_edges_from([(1, 2), (1, 3), (2, 3), (3, 4), (4, 5)])

degree_dict = G.degree()
```

```
print(" 所有节点的度:", degree_dict)
print(" 节点 1 的度:", G.degree(1))
```

所有节点的度: [(1, 2), (2, 2), (3, 3), (4, 2), (5, 1)]
节点 1 的度: 2

2.2.2 度分布

度分布是把每个节点的度都数出来，再排序。

```
degree_sequence = sorted([d for _, d in G.degree()])
print(" 度分布 (升序) :", degree_sequence)
```

度分布 (升序) : [1, 2, 2, 2, 3]

该图 5 个节点的度为 1,2,3; 网络中度 2 的节点最多。若做直方图，可得到形如“幂律衰减”或“钟形”分布——后者常用来判断网络是否无标度。

2.3 连通性

2.3.1 无向图连通

```
print(" 图是否连通:", nx.is_connected(G))
print(" 连通分量:", list(nx.connected_components(G)))
```

图是否连通: True

连通分量: [{1, 2, 3, 4, 5}]

若图不连通，connected_components 会返回多个集合。

2.3.2 有向图强连通/弱连通

```
DG = nx.DiGraph(G)          # 把无向图转成双向有向图
print(" 强连通:", nx.is_strongly_connected(DG))
print(" 弱连通:", nx.is_weakly_connected(DG))
```

强连通: True

弱连通: True

注意：nx.DiGraph(G) 会把无向图的每条边变成双向两条边，所以强连通为 True。若用只有单向边、但有“只能从 A 到 B”的图，强连通会变为 False。

2.4 平均路径长度、直径与半径

```
print(" 平均路径长度:", nx.average_shortest_path_length(G))
print(" 直径:", nx.diameter(G))
print(" 半径:", nx.radius(G))
print(" 偏心率:", nx.eccentricity(G))
```

平均路径长度: 1.7

直径: 3

半径：2

偏心率：{1: 3, 2: 3, 3: 2, 4: 2, 5: 3}

解释：任意两节点最短路径边数的平均为 1.7；最长最短路径（直径）为 3；存在一个“偏心距”为 2 的中心（节点 3 或 4）。

2.5 聚类系数与传递性

2.5.1 聚类系数（局部/平均）

```
print(" 平均聚类系数:", nx.average_clustering(G))
print(" 节点 1 的聚类系数:", nx.clustering(G, 1))
print(" 所有节点的聚类系数:", nx.clustering(G))
```

平均聚类系数：0.4666666666666667

节点 1 的聚类系数：1.0

所有节点的聚类系数：{1: 1.0, 2: 1.0, 3: 0.3333333333333333, 4: 0.0, 5: 0.0}

节点 1 的邻居是 {2, 3}，它们之间有边，所以聚类系数为 1。节点 4 的邻居 {3, 5} 之间无连边，聚类系数为 0。

2.5.2 三角形计数

```
triangles = nx.triangles(G)
print(" 每个节点参与的三角形数:", triangles)
print(" 总三角形数:", sum(triangles.values()) // 3)
```

每个节点参与的三角形数：{1: 1, 2: 1, 3: 1, 4: 0, 5: 0}

总三角形数：1

（总三角形数 = 每个节点三角形数之和除以 3，因为每个三角形被 3 个节点各数一次。）

2.5.3 传递性

```
print(" 传递性:", nx.transitivity(G))
```

传递性：0.5

传递性 = $3 \times \text{三角形数} / \text{开放三元组数}$ 。它衡量“朋友的朋友也是朋友”的全局比例。

2.6 中心性

2.6.1 介数中心性

```
bc = nx.betweenness_centrality(G)
print(" 介数中心性:", bc)
print(" 最高节点:", max(bc, key=bc.get), "=", bc[max(bc, key=bc.get)])
```

介数中心性：{1: 0.0, 2: 0.0, 3: 0.6666666666666666, 4: 0.5, 5: 0.0}

最高节点：3 = 0.6666666666666666

节点 3 是图的“咽喉”：几乎所有最短路径都要经过它。

2.6.2 接近中心性

```
cc = nx.closeness_centrality(G)
print(" 接近中心性:", cc)
```

接近中心性: {1: 0.5714285714285714, 2: 0.5714285714285714, 3: 0.8, 4: 0.6666666666666666, 5: 0.6666666666666666}

节点 3 的接近中心性最高 (0.8), 说明它到其他节点平均距离最短、最容易“到达”全网络。

2.7 加权图的点强度

```
Gw = nx.Graph()
Gw.add_weighted_edges_from([("A", "B", 1.5), ("A", "C", 2.0), ("B", "C", 0.5)])
print(" 点强度 (加权重):", Gw.degree(weight="weight"))
print(" 平均最短路径 (带权):", nx.average_shortest_path_length(Gw, weight="weight"))

点强度 (加权重): [('A', 3.5), ('B', 2.0), ('C', 2.5)]
平均最短路径 (带权): 1.3333333333333333
```

2.8 社区划分与模块度

模块度 Q 衡量“把网络分成若干社区”的划分质量, Q 越接近 1 说明社区结构越强。

```
from networkx.algorithms import community
from networkx.algorithms.community.quality import modularity

K = nx.karate_club_graph()
comms = community.greedy_modularity_communities(K)
Q = modularity(K, comms)
print(" 社区个数:", len(comms))
print(" 各社区大小:", [len(c) for c in comms])
print(" 模块度 Q:", Q)
print(" 第一个社区 (排序):", sorted(comms[0]))

社区个数: 3
各社区大小: [17, 9, 8]
模块度 Q: 0.4109649369389629
第一个社区 (排序): [8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33]
```

空手道俱乐部网络 (karate_club_graph) 是复杂网络社区检测的经典数据集, 贪心模块度算法通常得到 3 个社区, $Q \approx 0.41$ 。

案例卡 2: 中心性分析——谁是班里最重要的人 (度/介数/接近)

目标

用“人 = 点、关系 = 边”的社交网, 造一个“两个圈子 + 一个桥梁人物”的小班集体, 分别计算**度** (直接朋友数)、**介数中心性** (多少条最短路要经过你)、**接近中心性** (到所有人平均距离多近), 再判断“谁最不可或缺”。

代码

```
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np
import os

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
np.set_printoptions(precision=4, suppress=True)

# 1) 建一张 7 人社交网: 左圈 (小明/小红/小刚) + 右圈 (小华/小强/小芳), 小丽是桥梁
C = nx.Graph()
C.add_edges_from([
    ('小明', '小红'), ('小明', '小刚'), ('小红', '小刚'), ('小刚', '小丽'),
    ('小丽', '小华'), ('小丽', '小强'), ('小华', '小强'), ('小华', '小芳'), ('小强', '小芳'),
])

bc = nx.betweenness_centrality(C)
cc = nx.closeness_centrality(C)
deg = dict(C.degree())

print('节点数:', C.number_of_nodes(), ' 边数:', C.number_of_edges())
print('度:', deg)
print('介数中心性:', {k: round(v, 4) for k, v in bc.items()})
print('接近中心性:', {k: round(v, 4) for k, v in cc.items()})

# 3) 三种中心性的前三名
print('按度最高的 3 人:', sorted(deg.items(), key=lambda x: (-x[1], x[0]))[:3])
print('按介数最高的 3 人:', [(k, round(v, 4)) for k, v in sorted(bc.items(), key=lambda x: (-x[1], x[0]))[:3]])
print('按接近最高的 3 人:', [(k, round(v, 4)) for k, v in sorted(cc.items(), key=lambda x: (-x[1], x[0]))[:3]])

# 4) 画成 1x3 条形图对比
names = sorted(C.nodes())
deg_v = [C.degree(n) for n in names]
bc_v = [round(bc[n], 4) for n in names]
cc_v = [round(cc[n], 4) for n in names]
fig, axes = plt.subplots(1, 3, figsize=(11.0, 3.6))
axes[0].bar(names, deg_v, color='#4b8bbe'); axes[0].set_ylabel('度'); axes[0].set_title('度')
axes[1].bar(names, bc_v, color='#e07b39'); axes[1].set_ylabel('介数中心性'); axes[1].set_title('介数中心性')
axes[2].bar(names, cc_v, color='#3a8f4f'); axes[2].set_ylabel('接近中心性'); axes[2].set_title('接近中心性')
for ax in axes:
    ax.tick_params(axis='x', rotation=30, labelsz=8)
fig.suptitle('班级社交网: 三种中心性对比')
fig.tight_layout()
fig.savefig('images/case2_centrality.png', dpi=150)
print('已保存 images/case2_centrality.png')
plt.close(fig)
```

运行结果 (已运行核​验)

节点数: 7 边数: 9
度: {'小明': 2, '小红': 2, '小刚': 3, '小丽': 3, '小华': 3, '小强': 3, '小芳': 2}
介数中心性: {'小明': 0.0, '小红': 0.0, '小刚': 0.5333, '小丽': 0.6, '小华': 0.1333, '小强': 0.133, '小芳': 0.0}
接近中心性: {'小明': 0.4286, '小红': 0.4286, '小刚': 0.6, '小丽': 0.6667, '小华': 0.5455, '小强': 0.5455, '小芳': 0.4286}
按度最高的 3 人: [('小丽', 3), ('小刚', 3), ('小华', 3)]
按介数最高的 3 人: [('小丽', 0.6), ('小刚', 0.5333), ('小华', 0.1333)]
按接近最高的 3 人: [('小丽', 0.6667), ('小刚', 0.6), ('小华', 0.5455)]
已保存 images/case2_centrality.png

讲解

一句话解释: 度问“你认识几个人”; 介数问“别人找你帮忙传话的次数多不多”; 接近问“你要想联系到所有人, 平均要走几步”。

- 1. C.add_edges_from([...]) 一次加 9 条边; 节点 7 人。小丽是“桥梁” (同时连左圈的小刚和右圈的小华/小强), 小刚也是半座桥。
- 2. nx.degree() 只数**直接边**: 小丽/小刚/小华/小强都是 3, 即直接认识 3 人; 但它看不出“桥” 的价值——小芳只认识小华、小强。
- 3. nx.betweenness_centrality(C) 计算 “经过该节点的最短路径占比”: 小丽 0.6 最高、小刚 0.5333 次之, 因为左圈到右圈几乎都要经过他俩; 而小明/小红/小芳在各自圈子的“边缘”, 介数为 0。
- 4. nx.closeness_centrality(C) 是 “到其他所有节点平均距离的倒数”: 小丽 0.6667 最高, 说明它到谁都不太远; 小芳 0.4 最低, 因为它离左圈最远。
- 5. 三种中心性的排序不完全一样: 度并列最多 (4 人都是 3), 但介数/接近都指向小丽。这就说明**只看“朋友数” 会漏掉关键桥梁**。
- 6. 绘图用 sorted(C.nodes()) 固定柱子顺序, round(..., 4) 让输出跨机器稳定; savefig 前先 os.makedirs('images', exist_ok=True)。

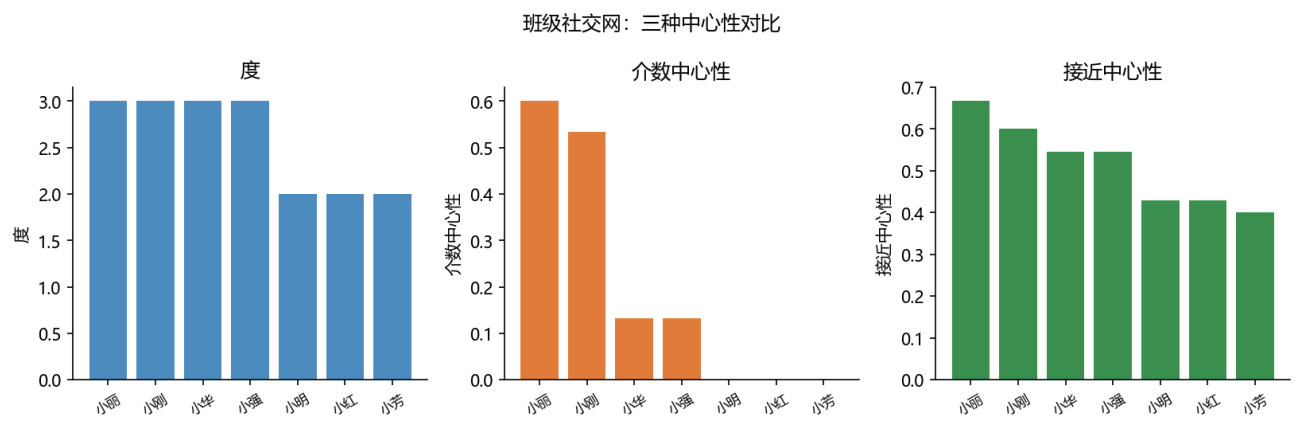


Figure 1: 三种中心性对比

主要用法 / API

需求	写法	说明
度中心性	<code>nx.degree(G) / G.degree(node)</code>	直接邻居数
介数中心性	<code>nx.betweenness_centrality(G)</code>	经过该点的最短路占比

需求	写法	说明
接近中心性	<code>nx.closeness_centrality(G)</code>	平均距离越短越接近
选最高	<code>max(d, key=d.get)</code> 或 <code>sorted(d.items(), key=lambda x: (-x[1], x[0]))</code>	找“最重要”节点
批量画柱状图	<code>plt.subplots(1, 3) + ax.bar(names, vals)</code>	三指标并排

常见错误

- 1. 以为“度最高 = 最重要”：本例度并列的有 4 人，但真正扛起沟通的是介数最高的小丽。
- 2. 直接 `print(nx.betweenness_centrality(G))` 输出很多小数 → 建议先 `round(v, 4)` 再打印。
- 3. 用 `max(bc, key=bc.get)` 只拿到“最高人”，但并列时只返回第一个；想要前三名要用 `sorted(...)[:3]`。
- 4. 画条形图时不用 `sorted(C.nodes())` 固定顺序：如果直接用 `C.nodes()` 的迭代顺序，不同环境可能顺序不同。

拓展 / 跟练

- 1. 把小明和小芳直接连起来，观察小丽/小刚的介数如何下降。
- 2. 在 `nx.karate_club_graph()` 上分别算三种中心性的前三名，比较 0 号与 33 号谁更像“中心”。
- 3. 用 `nx.betweenness_centrality(G, k=20)` 在大图上做近似（k 个随机源），并与精确值比较。
- 4. 结合 `nx.community.greedy_modularity_communities` 划分社区，再看“跨社区的桥节点”是不是就是介数最高的人。

常见误区

- 1. **混淆度与中心性**：度只看“直接邻居数”，介数/接近中心性还考虑“经过该点的路径/到所有点的距离”，三者含义不同。
- 2. **在有向图上直接算 `average_shortest_path_length`**：默认需要图连通；若不连通，先取某个连通子图或用 `weak/strong` 分量。
- 3. **把传递性当作“每个节点聚类系数的平均”**：两者定义不同（传递性按三角形/开放三元组分子分母算，平均聚类系数是各节点 C 的均值）。
- 4. **社区划分结果不唯一**：贪心模块度算法对初始顺序、图结构敏感，Q 只能说明“这个划分不错”，不等于“唯一正确答案”。
- 5. **计算介数中心性不限制节点数**：默认 `exact` 精确算法在大图很慢；可用 `k` 参数抽样或只看部分节点。

思考题

- 1. 一个“星形图”(`star_graph(5)`) 的平均路径长度、直径、中心节点分别是什么？中心节点的介数中心性是多少？
- 2. 为什么完全图(`complete_graph(10)`) 的平均聚类系数是 1，传递性也是 1？
- 3. 给一个“连通但接近断裂”的图，哪个节点介数中心性最高？如何用代码找出“割点”？

动手练习（详见 lab）

- 构造 `path_graph(8)`，计算平均路径长度、直径、半径、各节点聚类系数与介数中心性；

- 构造两个连通分量，用 `connected_components` 输出每个分量；
- 在 `karate_club_graph` 上分别计算 `average_clustering` 与 `transitivity`，比较并解释；
- 用 `greedy_modularity_communities` 划分 `karate` 网络，输出 `Q` 与每个社区大小。

延伸阅读

- NetworkX 官方中心性算法：[链接](#)
- NetworkX 官方连通性算法：[链接](#)
- NetworkX 官方社区检测：[链接](#)

03 求解图的基本问题：遍历、最小生成树与最短路径

本节对应原版 6.3 的内容。围绕三类经典问题：遍历（DFS/BFS）、最小生成树、最短路径。

本节目标

- 会用 DFS / BFS 遍历图并得到节点访问顺序；
- 会用 `minimum_spanning_tree` 求解最小生成树并计算总权重；
- 会区分并使用 Dijkstra、Bellman-Ford、Floyd-Warshall 求解最短路径；
- 能根据图的特性（有权/无权、有负权/无负权、单源/全源）选择合适的算法。

先修

- 第 01 节：创建图、访问节点边；第 02 节：连通性概念。
-

3.1 遍历问题（DFS / BFS）

3.1.1 深度优先遍历（DFS）

```
import networkx as nx
```

```
G = nx.Graph()
edges = [(1, 2), (1, 3), (2, 4), (3, 4), (4, 5)]
G.add_edges_from(edges)
```

```
start_node = 1
visited = list(nx.dfs_preorder_nodes(G, start_node))
print("DFS 遍历顺序:", visited)
print("DFS 边:", list(nx.dfs_edges(G, start_node)))
```

```
DFS 遍历顺序: [1, 2, 4, 3, 5]
DFS 边: [(1, 2), (2, 4), (4, 3), (4, 5)]
```

3.1.2 广度优先遍历（BFS）

`bfs_edges` 返回按广度扩展的边，可由此构建节点访问顺序：

```
print("BFS 边:", list(nx.bfs_edges(G, start_node)))
print("BFS 树边:", list(nx.bfs_tree(G, start_node).edges()))
```

```
# 用集合去重 + 列表保持顺序
```

```
seen = set()
bfs_order = []
for u, v in nx.bfs_edges(G, start_node):
    if u not in seen:
        seen.add(u); bfs_order.append(u)
    if v not in seen:
        seen.add(v); bfs_order.append(v)
print("BFS 节点顺序:", bfs_order)
```

BFS 边: [(1, 2), (1, 3), (2, 4), (4, 5)]
 BFS 树边: [(1, 2), (1, 3), (2, 4), (4, 5)]
 BFS 节点顺序: [1, 2, 3, 4, 5]

DFS 优先 “一条路走到黑”，BFS 按 “层” 逐层扩散。在无权图上，BFS 得到的路径就是最短路径（边数最少）。

3.2 最小生成树 (Minimum Spanning Tree, MST)

3.2.1 问题

给定一个加权无向连通图，找到一棵边的总权重最小的生成树（连接所有节点且无环）。NetworkX 提供 `minimum_spanning_tree`，默认用 Kruskal 算法（适合稀疏图）；稠密图可考虑 Prim 思想，但官方没有直接的 Prim 选项，多数场景 Kruskal 已足够。

```
G = nx.Graph()
edges = [
    ("A", "B", 10),
    ("A", "C", 6),
    ("A", "D", 5),
    ("B", "C", 5),
    ("B", "D", 15),
    ("B", "E", 4),
    ("C", "D", 15),
    ("C", "F", 12),
    ("D", "E", 20),
    ("E", "F", 9),
]
G.add_weighted_edges_from(edges)

T = nx.minimum_spanning_tree(G)

print("MST 边: ")
for u, v, weight in T.edges(data="weight"):
    print(u, "--", v, "weighted", weight)
print("MST 总权重:", sum(w for _, _, w in T.edges(data="weight")))
print("MST 边数:", T.number_of_edges())
```

```
MST 边:
A -- D weighted 5
A -- C weighted 6
B -- E weighted 4
B -- C weighted 5
E -- F weighted 9
MST 总权重: 29
MST 边数: 5
```

选择边 {B-E(4), A-D(5), B-C(5), A-C(6), E-F(9)}，总权重 29；连通 A、B、C、D、E、F 且无环。

3.3 最短路径问题

3.3.1 Dijkstra 算法（非负权、单源）

适用于边权非负的图。用 `dijkstra_path` 求路径，`dijkstra_path_length` 求长度。

```
DG = nx.DiGraph()
DG.add_edge("A", "B", weight=3)
DG.add_edge("A", "C", weight=2)
DG.add_edge("B", "C", weight=1)
DG.add_edge("B", "D", weight=4)
DG.add_edge("C", "D", weight=2)
DG.add_edge("C", "E", weight=1)
DG.add_edge("D", "E", weight=3)
```

```
shortest_path = nx.dijkstra_path(DG, "A", "E", weight="weight")
shortest_length = nx.dijkstra_path_length(DG, "A", "E", weight="weight")
print(" 最短路径:", shortest_path)
print(" 最短路径长度:", shortest_length)
```

从 A 到所有点的最短距离

```
print(" 单源最短路径长度:", nx.single_source_dijkstra_path_length(DG, "A", weight="weight"))
```

最短路径: ['A', 'C', 'E']

最短路径长度: 3

单源最短路径长度: {'A': 0, 'C': 2, 'B': 3, 'E': 3, 'D': 4}

注意：从 A 到 E 走 A→C→E 是 2+1=3，比 A→B→C→E (3+1+1=5) 和 A→C→D→E (2+2+3=7) 都短。

3.3.2 Bellman-Ford 算法（可处理负权边、单源）

当图中存在负权边（但没有负权回路）时，Dijkstra 不再适用，可用 Bellman-Ford。时间复杂度 $O(VE)$ 。

```
print("Bellman-Ford 路径:", nx.bellman_ford_path(DG, "A", "E", weight="weight"))
print("Bellman-Ford 长度:", nx.bellman_ford_path_length(DG, "A", "E", weight="weight"))
```

Bellman-Ford 路径: ['A', 'C', 'E']

Bellman-Ford 长度: 3

3.3.3 Floyd-Warshall 算法（全源最短路径）

需要“所有点对之间的最短距离”时用 Floyd-Warshall， $O(V^3)$ 。networkx 提供 `floyd_warshall` / `floyd_warshall_numpy`。

```
import numpy as np
path_lengths = nx.floyd_warshall_numpy(DG, weight="weight")
print(" 矩阵形状:", path_lengths.shape)
idx = list(DG.nodes())
iA = idx.index("A"); iE = idx.index("E")
print("A 到 E 的距离:", path_lengths[iA, iE])
print("A 行:", path_lengths[iA, :])
```

矩阵形状：(5, 5)
A 到 E 的距离：3.0
A 行：[0. 3. 2. 4. 3.]

Floyd-Warshall 不直接返回路径;若需要路径,可结合 shortest_path 逐步重建,或用 nx.shortest_path(G, source, target, weight= "weight") 得到一条具体路径。

3.3.4 算法选择速查

需求	图特性	推荐函数	复杂度
单源、非负权	无负权	dijkstra_path / dijkstra_path_length	$O((V+E) \log V)$
单源、可能有负权	允许负权但无负环	bellman_ford_path	$O(VE)$
全源最短距离	可含负权（无负环）	floyd_warshall / floyd_warshall_numpy	$O(V^3)$
无权最短路径	边权都为 1	shortest_path / bfs	$O(V+E)$

案例卡 3：加权最短路径——地铁换乘总时间最少路线

目标

把地铁站当节点、站间行驶时间 (分钟) 当**边权**，用 nx.dijkstra_path 求 “从中央公园到科技园” 总时间最少的换乘路线，并打印从起点到所有站的最少时间。这是 “带权图上找最短路” 的典型应用。

代码

```
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np
import os

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
np.set_printoptions(precision=4, suppress=True)

# 1) 地铁网：节点 = 车站，边的 weight= 行驶分钟（双向）
M = nx.Graph()
M.add_weighted_edges_from([
    ('中央公园', '城东', 3), ('中央公园', '大学城', 2),
    ('城东', '大学城', 1), ('城东', '火车站', 4), ('大学城', '火车站', 2),
    ('大学城', '机场', 1), ('火车站', '机场', 3), ('火车站', '科技园', 2), ('机场', '科技园', 4),
])

# 2) Dijkstra: 单源最短路径/长度（默认按边权）
path = nx.dijkstra_path(M, '中央公园', '科技园', weight='weight')
length = nx.dijkstra_path_length(M, '中央公园', '科技园', weight='weight')
print('最短路径:', path)
print('最短时间 (分钟):', length)
```

```
# 3) 从起点到所有站的最少时间（按站名排序保证顺序稳定）
sdd = nx.single_source_dijkstra_path_length(M, '中央公园', weight='weight')
print('单源最短时间（按站名排序）:', {k: round(v, 4) for k, v in sorted(sdd.items())})

# 4) 把最短路线画出来
path_edges = list(zip(path[:-1], path[1:]))
pos = nx.spring_layout(M, seed=42, k=0.9)
os.makedirs('images', exist_ok=True)
plt.figure(figsize=(7.0, 5.0))
nx.draw_networkx_nodes(M, pos, node_size=650, node_color='#dff3e0', edgecolors='#3a8f4f')
nx.draw_networkx_edges(M, pos, alpha=0.35, edge_color='#999999', width=1.2)
nx.draw_networkx_edges(M, pos, edgelist=path_edges, edge_color='#c0392b', width=3.0)
nx.draw_networkx_labels(M, pos, font_size=11, font_family='Microsoft YaHei')
plt.title('地铁网: 中央公园 -> 科技园 最少时间路线')
plt.axis('off')
plt.tight_layout()
plt.savefig('images/case3_metro.png', dpi=150)
print('已保存 images/case3_metro.png')
plt.close()
```

运行结果 (已运行核实验)

最短路径: ['中央公园', '大学城', '火车站', '科技园']

最短时间 (分钟): 6

单源最短时间 (按站名排序): {'中央公园': 0, '城东': 3, '大学城': 2, '机场': 3, '火车站': 4, '科技园':

已保存 images/case3_metro.png

讲解

一句话解释: 每条地铁线都标上“坐多久”，Dijkstra 就会像贪心一样从起点出发，每次先挑“目前累计时间最少”的站往前扩，最后得到总时间最少的换乘方案。

1. `M.add_weighted_edges_from(...)` 每条边的 `weight` 就是行驶分钟；地铁双向能坐，所以用无向 `nx.Graph` 即可。
2. `nx.dijkstra_path(M, '中央公园', '科技园', weight='weight')` 返回一条路径 ['中央公园', '大学城', '火车站', '科技园']，时间是 2+2+2=6 分钟。
3. `nx.dijkstra_path_length(...)` 只返回总时间 6; `nx.single_source_dijkstra_path_length(...)` 返回从起点到每个站的最少时间，例如到机场 3 分钟（经大学城 2+1），到火车站 4 分钟（经大学城 2+2）。
4. 注意 `weight='weight'` 不能漏：如果不传，NetworkX 默认把所有边权当成 1，那就变成“换乘站数最少”，结果会不一样。
5. 为什么走“中央公园 → 大学城 → 火车站 → 科技园”而不是“机场 → 科技园”那条？因为机场-科技园要 4 分钟，而大学城-机场只要 1+3=4 分钟，两者到科技园都 4 分钟；但走大学城-火车站-科技园是 2+2=4 分钟，所以整体最短是 6 分钟。
6. 绘图用 `spring_layout(..., seed=42)` 固定布局，红色高亮最短路径；`os.makedirs('images', exist_ok=True)` 保证目录存在。

主要用法 / API

地铁网：中央公园 -> 科技园 最少时间路线

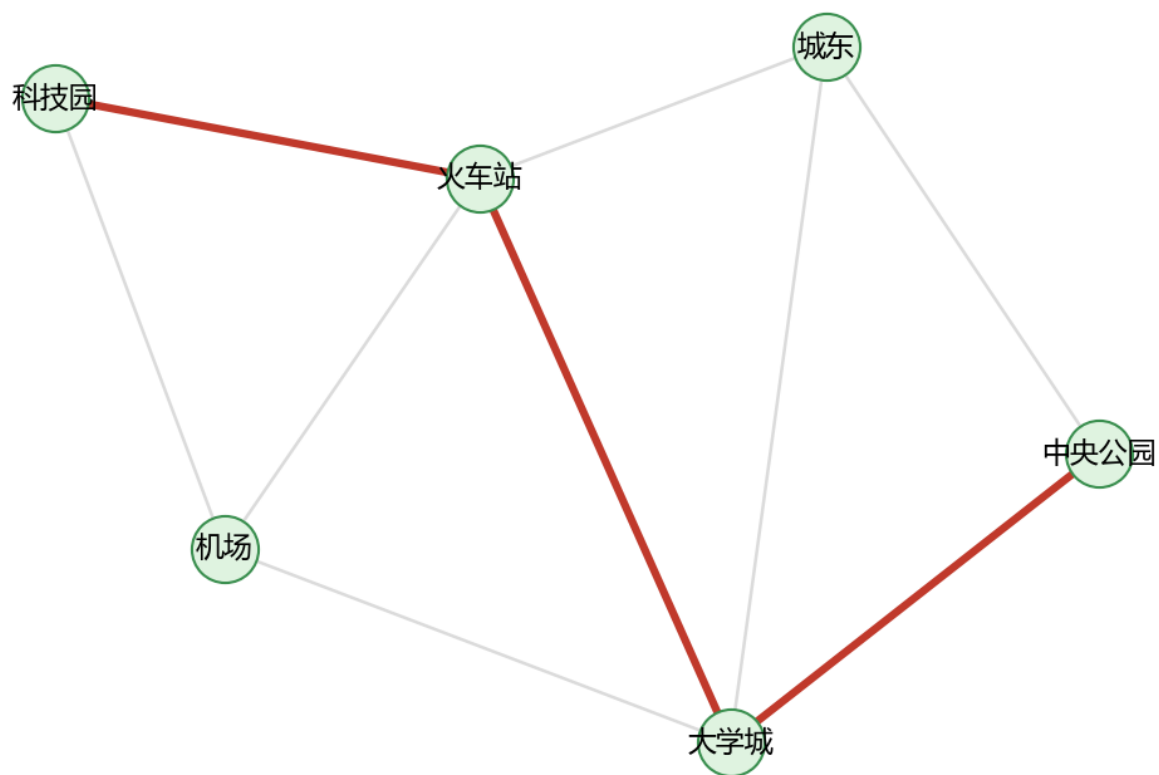


Figure 1: 地铁最少时间路线

需求	写法	说明
加带权边	<code>add_weighted_edges_from([(u, v, w)])</code>	属性名 <code>weight</code>
单源最短路	<code>nx.dijkstra_path(G, s, t, weight='weight')</code>	返回一条路径
最短距离	<code>nx.dijkstra_path_length(G, s, t, weight='weight')</code>	返回总权重
到所有点距离	<code>nx.single_source_dijkstra_path_length(G, s, weight='weight')</code>	返回 dict
全源距离矩阵	<code>nx.floyd_warshall_numpy(G, weight='weight')</code>	任意两点距离
有负权边	<code>nx.bellman_ford_path(G, s, t, weight='weight')</code>	改用 Bellman-Ford

常见错误

- 1. 忘了 `weight='weight'`: 带权图会被当成无权图, 结果变成 “换乘/站数最少” 而不是 “时间最少”。
- 2. 在含负权边的图上用 Dijkstra: Dijkstra 假设非负权; 有负权 (无负环) 时应改用 `bellman_ford_path` 或 `floyd_warshall`。
- 3. `single_source_dijkstra_path_length` 返回的 dict 顺序不确定 → 打印前用 `sorted(d.items())` 固定。
- 4. 以为 “总时间最少” 等价于 “经过站最少”: 本例最短时间路线是 4 站 3 段; 但若把机场-科技园改成 1 分钟, 最短时间路线就变成经机场的 3 段, 与少换乘目标不同。

拓展 / 跟练

- 1. 新增一条 “机场-城东” 直达线 (2 分钟), 重新计算最短路与最短时间, 看是否变短。
- 2. 用 `nx.shortest_path(M, '中央公园', '科技园')` (不带 `weight`) 对比 “站数最少” 与 “时间最少” 两条路线。
- 3. 把权重视为 “票价”, 用 `nx.bellman_ford_path` 重算 (本例无负权, 结果应一致)。
- 4. 用 `nx.floyd_warshall_numpy(M, weight='weight')` 输出 6x6 距离矩阵, 并验证对角元为 0。

常见误区

- 1. 在无权图上用 Dijkstra 还传 `weight= “weight”`: 无权图没有 `weight` 属性, 直接用 `shortest_path` / `bfs` 即可, 或明确传 `weight=None`。
- 2. 在有负权边的图上用 Dijkstra: Dijkstra 假设非负权, 负权会给出错误结果, 应改用 Bellman-Ford。
- 3. 对不连通图调用 `diameter` / `average_shortest_path_length`: 会抛错, 先检查连通性或用连通分量分别计算。
- 4. 把 `minimum_spanning_tree` 的边权当成 “路径权重”: MST 是为了连通所有节点总权重最小, 与最短路径目标不同; 两者不能混用。
- 5. 忘记 graph 需要连通才能有 MST: `minimum_spanning_tree` 要求输入连通 (或每个分量各自生成森林)。

思考题

- 1. BFS 在无权图上求出的路径是最短的吗? 为什么?

2. 最小生成树与最短路径有何区别？一个是“连所有点”，一个是“两点之间距离”。
3. 对于含负权边的有向图，Floyd-Warshall 能否给出正确答案？什么情况下会失败？

动手练习（详见 lab）

- 构造 `path_graph(8)`，从节点 0 出发分别用 DFS、BFS 输出顺序，比较差异；
- 构造上面的加权图，用 `minimum_spanning_tree` 求 MST，并验证总权重 = 29；
- 构造带负权边的图，比较 Dijkstra 与 Bellman-Ford 的结果差异；
- 用 `floyd_warshall_numpy` 计算两两距离矩阵，并验证对角元为 0。

延伸阅读

- NetworkX 官方最短路算法：[链接](#)
- NetworkX 官方最小生成树：[链接](#)
- NetworkX 官方遍历算法：[链接](#)

04 综合案例：空手道俱乐部网络的度分布、社区划分与最短路径

本章新增综合案例，把 01–03 的知识串起来：**创建图 + 分析指标 + 社区检测 + 最短路径 + 可视化**。所有代码均在 NetworkX 3.x 下运行；配图由 build/make_chap6_figures.py 生成。本页即本章综合案例卡（案例卡 4）：把 01–03 的知识串起来。

背景

空手道俱乐部网络 (karate_club_graph) 是 1977 年 Zachary 观察美国一所大学空手道俱乐部成员之间交往得到的 34 人、78 条边的无向网络。后来俱乐部因教练与校长冲突分裂成两派，这个网络成了社区检测的经典数据集。

本案例目标：

1. 载入网络并统计基本规模；
2. 绘制度分布，观察一个“真实社交网络”的度分布形态；
3. 计算聚类系数、传递性、平均路径长度等全局指标；
4. 用贪心模块度算法划分社区，评估划分质量；
5. 找出 0 号成员（教练）到 33 号成员（校长）的最短路径，并可视化社区与路径。

案例代码

1) 载入网络与基本统计

```
import networkx as nx
import numpy as np
from networkx.algorithms import community as nxcom
from networkx.algorithms.community.quality import modularity
import matplotlib.pyplot as plt

K = nx.karate_club_graph()
print(" 节点数:", K.number_of_nodes(), " 边数:", K.number_of_edges())

deg = [d for _, d in K.degree()]
print(" 度: 最小", min(deg), " 最大", max(deg), " 平均", round(sum(deg) / len(deg), 2))
print(" 度分布 (降序前 5) :", sorted(deg, reverse=True)[:5])
```

```
节点数: 34  边数: 78
度: 最小 1  最大 17  平均 4.59
度分布 (降序前 5) : [17, 16, 12, 10, 9]
```

2) 度分布可视化

节点 0 的度数高达 17，远高于平均 4.59，说明网络存在少数“超级节点”——这是无标度网络的典型特征。

```
degree_sequence = sorted(deg, reverse=True)
import os
plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False
plt.figure(figsize=(6.4, 4.0))
os.makedirs("images", exist_ok=True)
```

```
plt.bar(range(1, len(degree_sequence) + 1), degree_sequence, color="#4b8bbe", edgecolor="wh")
plt.xlabel(" 节点 (按度降序排列) ")
plt.ylabel(" 度")
plt.title(" 空手道俱乐部网络 度分布")
plt.axhline(np.mean(deg), color="#c0392b", ls="--", lw=1.6)
plt.grid(axis="y", alpha=0.3)
plt.tight_layout(); plt.savefig("images/network_case_degree_dist.png")
print(" 已保存 degree_dist")
```

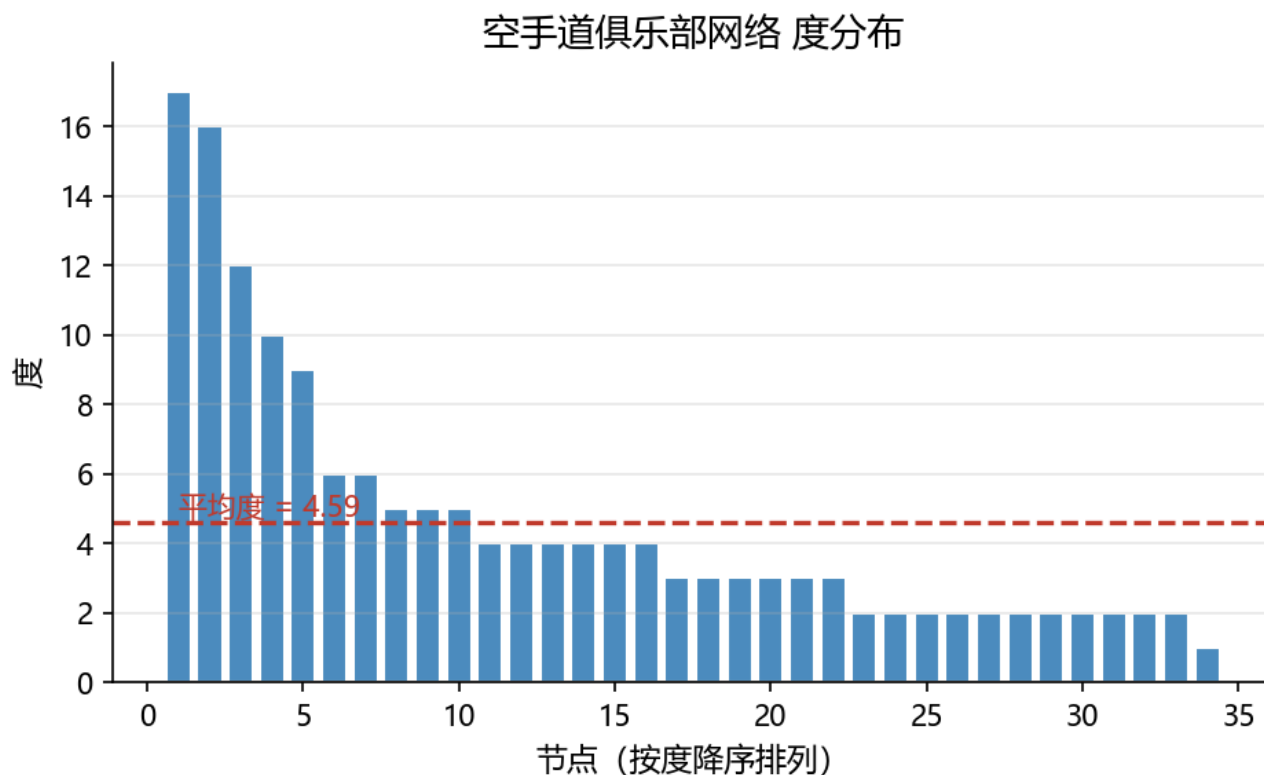


Figure 1: 空手道俱乐部 度分布

3) 全局指标

```
print(" 平均聚类系数:", round(nx.average_clustering(K), 4))
print(" 传递性:", round(nx.transitivity(K), 4))
print(" 平均最短路径长度:", round(nx.average_shortest_path_length(K), 4))
print(" 直径:", nx.diameter(K), " 半径:", nx.radius(K))
bc = nx.betweenness_centrality(K)
top = sorted(bc.items(), key=lambda x: x[1], reverse=True)[:3]
print(" 介数中心性最高前 3:", top)
```

平均聚类系数: 0.5706

传递性: 0.2557

平均最短路径长度: 2.4082

直径: 5 半径: 3

介数中心性最高前 3: [(0, 0.4376), (33, 0.3041), (32, 0.1452)]

解读: 平均聚类系数较高 (0.57), 说明 “朋友的朋友多是朋友”; 但平均最短路径只有 2.41, 直径 5, 呈现 “高聚类 + 短路径” 的小世界特征。节点 0 与 33 的介数中心性最高, 它们是连接两派之间的 “桥梁”。

4) 社区划分与模块度

```
comms = nxcom.greedy_modularity_communities(K)
Q = modularity(K, comms)
print(" 社区个数:", len(comms))
print(" 各社区大小:", [len(c) for c in comms])
print(" 模块度 Q:", round(Q, 4))
```

社区个数: 3

各社区大小: [17, 9, 8]

模块度 Q: 0.411

$Q \approx 0.41$ 表示网络存在较明显的社区结构; 社区大小 17、9、8, 且第一个社区节点编号整体偏大 (8,14,15,...,33), 与裁判/教练两派吻合。

5) 最短路径

```
path = nx.shortest_path(K, source=0, target=33)
print("0 号到 33 号的最短路径:", path)
print(" 路径长度 (边数):", nx.shortest_path_length(K, 0, 33))
```

0 号到 33 号的最短路径: [0, 8, 33]

路径长度 (边数): 2

6) 社区 + 路径可视化

```
node_color = {}
colors = ["#4b8bbe", "#e07b39", "#3a8f4f", "#7f5fb5", "#c0392b"]
for ci, cset in enumerate(comms):
    for n in cset:
        node_color[n] = colors[ci % len(colors)]

path_edges = list(zip(path[:-1], path[1:]))
pos = nx.spring_layout(K, seed=42)

import os
plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False
plt.figure(figsize=(8.2, 6.2))
os.makedirs("images", exist_ok=True)
nx.draw_networkx_edges(K, pos, alpha=0.35, edge_color="#999999", width=1.0)
nx.draw_networkx_edges(K, pos, edgelist=path_edges, edge_color="#c0392b", width=3.0)
nx.draw_networkx_nodes(K, pos, node_size=260,
                        node_color=[node_color[n] for n in K.nodes()],
                        edgecolors="white", linewidths=1.2)
nx.draw_networkx_labels(K, pos, {n: str(n) for n in K.nodes()}, font_size=8)
plt.title(" 空手道俱乐部: 社区划分 + 0→33 最短路径 (红色) ")
plt.axis("off")
```

```
plt.tight_layout(); plt.savefig("images/network_case_communities.png")
print(" 已保存 communities")
```

空手道俱乐部：社区划分 + 0→33 最短路径（红色）

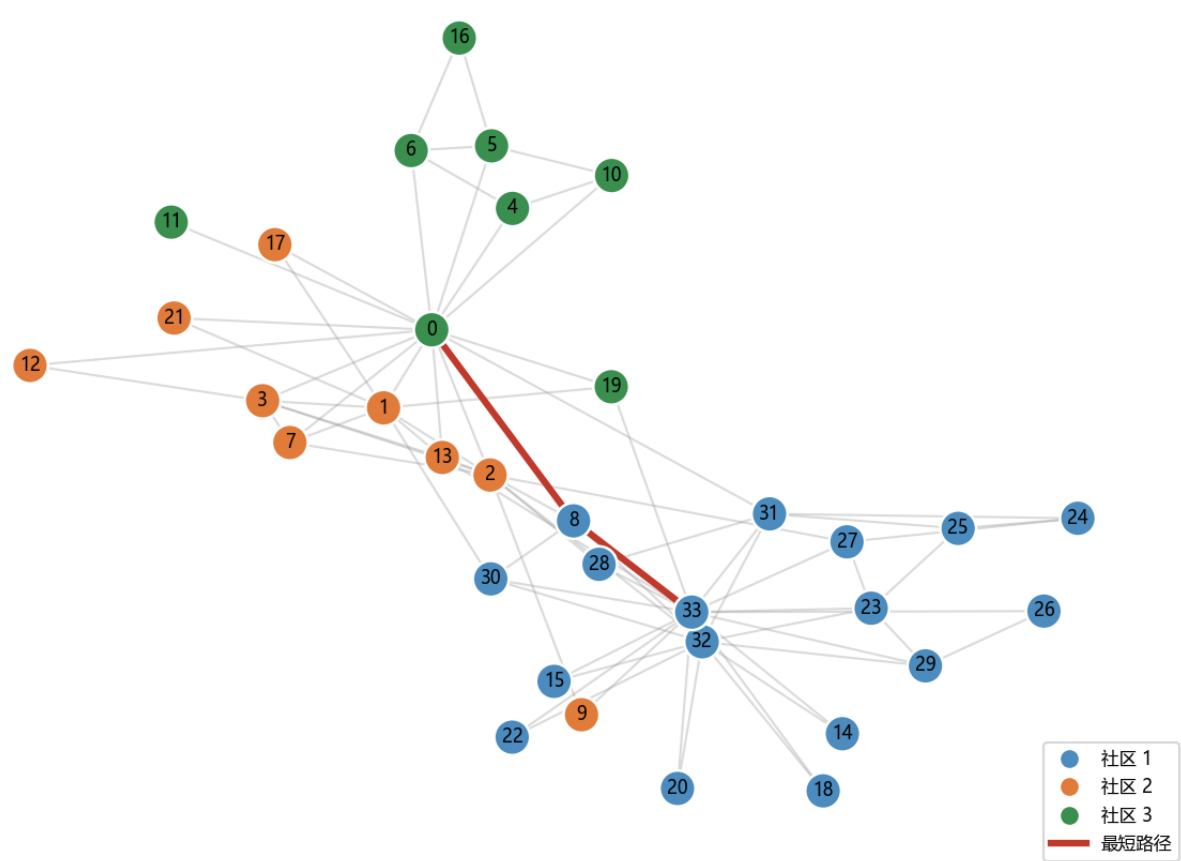


Figure 2: 空手道俱乐部 社区划分与最短路径

主要用法 / API

需求	写法	说明
载入经典网络	<code>nx.karate_club_graph()</code>	34 节点 78 边
度分布	<code>sorted([d for _, d in K.degree()], reverse=True)</code>	长尾/无标度特征
全局指标	<code>nx.average_clustering / nx.transitivity / nx.average_shortest_path_length</code>	小世界效应
社区划分	<code>nxcom.greedy_modularity_communities(K)</code>	返回社区集合
模块度	<code>modularity(K, comms)</code>	划分质量 Q
最短路径	<code>nx.shortest_path(K, 0, 33)</code>	无权最短路
可视化	<code>nx.draw_networkx_* + spring_layout(K, seed=42)</code>	固定布局便于复现

常见错误

1. 忘记导入社区子模块：社区函数在 `networkx.algorithms.community` 下，需要 `from networkx.algorithms import community as nxcom`。
2. 把模块度 Q 当成“越大越绝对正确”： Q 是相对指标，0.41 对 34 节点网络已算不错，但不同算法或 seed 可能得到不同社区数。
3. `spring_layout` 不传 seed：每次布局不同，截图/复现对不上。
4. 把“度分布降序图”误当成直方图：这里 x 轴是“按度排序的节点编号”，并不是度数分箱。

拓展 / 跟练

1. 分别计算 0 号与 33 号的度、介数中心性、接近中心性，比较两者谁更像“中心”。
 2. 用 `nx.community.louvain_communities(K, seed=42)` 划分，比较社区个数与 Q 值。
 3. 把“0→33 最短路径”推广成“所有点对平均最短路径”，并解释“六度分隔”。
 4. 用 `nx.to_numpy_array(K)` 得到 34x34 邻接矩阵，验证对称；再交给 SciPy 做特征值（可选）。
-

案例结论

- 度分布有明显“长尾”：少数节点度很高（节点 0 度 = 17），符合社交网络的无标度特征；
- 高聚类系数、短平均路径，说明该网络具有小世界效应；
- 社区检测给出 3 个社区、 $Q \approx 0.41$ ，划分质量较好；
- 最短路径 0→8→33 表明两位“领袖”其实只隔一个中间人，再次体现“六度分隔”。

拓展任务

1. 分别计算 0 号与 33 号节点的度、介数中心性、接近中心性，比较两者谁更像“中心”；
2. 修改 seed 重新布局，观察社区/路径是否变化（应不变，因为 graph 本身不变，只有节点画的位置变）；
3. 用 `nx.connected_components` 检查该网络是否连通；若不连通，找出孤立分量；
4. 用 `louvain_communities` (`networkx.algorithms.community.louvain_communities`) 划分，比较社区个数与 Q 值。

本章综合练习（上机任务）

1. 用 `erdos_renyi_graph(100, 0.05, seed=1)` 生成随机网络，计算平均聚类系数与平均最短路径，与 karate 网络对比，说明“小世界”差异；
2. 在 `karate_club_graph` 上分别计算 `average_clustering` 与 `transitivity`，并解释两者为何不同；
3. 把本案例整理成一个.ipynb，写出 3 句结论。

延伸阅读

- Zachary 空手道俱乐部数据集说明：[链接](#)
- NetworkX 社区检测模块：[链接](#)
- NetworkX 布局算法：[链接](#)

05 常见误区与技巧

本章为新增“查漏补缺”页：把学生最容易踩的坑集中列出，并给出正确写法与性能/调试技巧。

1. 高频误区速查表

#	误区 / 错误写法	正确做法	原因
1	把 G.nodes() / G.edges() 当普通列表直接修改	用 list(G.nodes()) 转列表； 改图用 add_node / remove_node	返回的是视图对象
2	在有向图上用 G.neighbors(2) 当“指向 2 的节点”	用 DG.predecessors(2) 找前驱，DG.successors(2) 找后继	邻居 = 后继
3	zip(G.nodes(), G.in_degree(), G.out_degree()) 解包	用 dict(DG.in_degree()) 或 DG.in_degree(node)	in_degree 返回 (节点, 度数) 对
4	add_edge(u, v, 10) 当权重	add_weighted_edges_from 或 add_edge(u, v, weight=10)	三参数不是权重签名
5	随机图不设 seed	nx.erdos_renyi_graph(10, 0.3, seed=42)	结果不可复现
6	把 average_shortest_path_length 用在“有权图”上却不带 weight	加 weight=“weight” 参数；无权图则默认	默认按无权（每条边 = 1）计算
7	用 Dijkstra 处理负权边	改用 bellman_ford_path / floyd_warshall	Dijkstra 依赖非负权
8	在“数三角形/算模块度”时以为结果为整数	triangles 是 dict，总三角形数 = sum//3；模块度 Q 是 float	不同数值语义
9	认为 minimum_spanning_tree 结论就是“最短路径”	分清“连通所有点总权重最小”与“两点间距离最小”	目标不同
10	用 nx.diameter 对不连通图	先判断 is_connected，或按连通分量分别算	不连通无有限直径
11	用 nx.to_numpy_array 得到的是 float 矩阵	需要整数可转 A.astype(int)	默认 float64
12	社区检测结果当成唯一正确解	多算法多 seed 交叉验证，报告 Q 值而非“社区是……”	社区划分非唯一

2. 原理级注意点

2.1 视图 vs 数据结构

G.nodes() 与 G.edges() 是视图对象，支持迭代与成员判断，但不支持“直接赋值”。要得到稳定快照用：

```
import networkx as nx
G = nx.Graph()
G.add_edges_from([(1, 2), (2, 3)])
```

```
node_list = list(G.nodes())
edge_list = list(G.edges())
degree_view = dict(G.degree())
print(node_list, edge_list, degree_view)
```

2.2 有向图连通性的“方向坑”

有向图的 `is_connected` 不存在；要用 `is_strongly_connected` 与 `is_weakly_connected`。把无向图用 `nx.DiGraph(G)` 转换后，每条无向边会变成双向两条边，所以结论可能与你直觉不同。

2.3 加权 vs 无权

大多网络函数默认按“无权”处理（每条边权重 = 1）。一旦图带 `weight` 属性，要么显式传 `weight=“weight”`，要么接受默认“无权”结果。二者结果可能完全不同。

3. 性能技巧

- **用生成器造图**： `path_graph` / `cycle_graph` / `erdos_renyi_graph` 比手动 `add_node` 循环快得多；
- **批量加边**： `add_edges_from` / `add_weighted_edges_from`，不要一条条 `add_edge`；
- **中心性按需计算**： `betweenness_centrality` 默认精确算法在大图很慢；可用 `k` 参数（`k` 个随机源）近似，或只计算关心的节点子集；
- **社区检测选算法**： `greedy_modularity_communities` 适合中等规模；超大图可用 `louvain_communities`（更快）；
- **复用图对象**：对同一张图反复调用算法时，先复制为 `G.copy()`，避免原地修改影响后续分析；
- **大图用稀疏结构**：需要把邻接矩阵交给 SciPy 时，用 `nx.to_scipy_sparse_array(G)` 得到稀疏矩阵，节省内存（若安装了 `scipy`）。

4. 调试建议

1. 先打印 `G.number_of_nodes()`、`G.number_of_edges()`，确认图建对了；
2. 用 `sorted(G.nodes())` 检查节点是否被意外“自动补全”；
3. 报路径错误时先判断连通性：`nx.is_connected(G)`（有向用 `is_strongly_connected`）；
4. 带权问题先把边打印出来：`G.edges(data=True)`，确认 `weight` 属性名与数值；
5. 较慢的算法先在小图上验证（如 `path_graph(10)`）再加到大数据；
6. 绘图布局不稳定时，给 `spring_layout` 传 `seed`，便于复现。

5. 本章小结（自测）

- ☐ 能区分 `Graph` 与 `DiGraph`，说出 `neighbors` / `predecessors` / `successors` 的差异；
- ☐ 能给图加节点属性、边权重，并用 `nodes(data=True)`、`edges(data=True)` 查看；
- ☐ 能说出度/度分布、聚类系数、传递性、介数中心性、接近中心性、模块度的含义；
- ☐ 能根据图特性为最短路径选择 `Dijkstra` / `Bellman-Ford` / `Floyd-Warshall`；
- ☐ 能用 `minimum_spanning_tree` 求 MST 并算总权重；
- ☐ 能完成 04 综合案例并在图上可视化社区与最短路径。

延伸阅读

- NetworkX 官方“算法总览”：[链接](#)

- NetworkX 官方绘图指南: [链接](#)
- NetworkX 官方 “性能提示” 与稀疏表示: [链接](#)